NOORUL ISLAM CENTRE FOR HIGHER EDUCATION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

APPENDIX: CONSOLIDATED CODE ARTIFACTS, ARCHITECTURES & EXECUTION TELEMETRY

Comprehensive Visual Evidence for Projects 1 through 6 (Chapters 3 to 8)

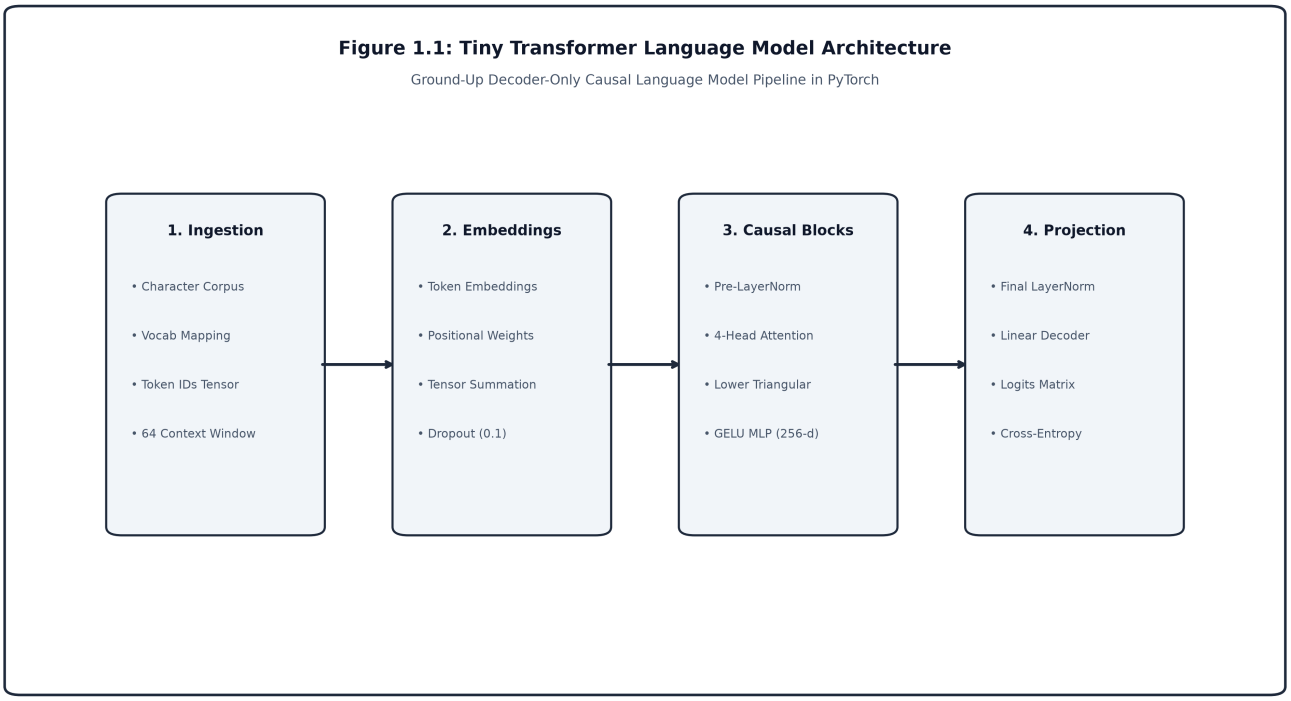
Student: **ADITHIYAN P V** | Register No: **[Insert Register Number]**

Appendix Overview: This appendix compiles all structural diagrams, core algorithm implementations, and runtime telemetry traces across all six mini-project systems. All figures are formatted in high-contrast monochrome (black text on pure white background) at 300 DPI publication standards, ensuring optimal legibility for print submission and formal academic evaluation.

CHAPTER 3 — PROJECT 1: TRANSFORMER-BASED LANGUAGE MODEL

Figure 1.1: Tiny Transformer Language Model Pipeline Architecture

Classification: *System Architecture Flow* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*



Technical Description: End-to-end dataflow pipeline of the causal decoder-only transformer model implemented in PyTorch, showing character corpus tokenization, dual embedding summation, multi-head causal blocks, and linear projection.

Figure 1.2: Tiny Transformer Core Architecture & Residual Block Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 1.2: Tiny Transformer Core Architecture PyTorch Code

```
class TransformerBlock(nn.Module):
    def __init__(self, emb_dim: int, num_heads: int, ffn_dim: int, drop: float):
        super().__init__()
        self.ln1 = nn.LayerNorm(emb_dim) # Pre-LayerNorm formulation
        self.attn = CausalSelfAttention(emb_dim, num_heads, drop)
        self.ln2 = nn.LayerNorm(emb_dim)
        self.mlp = nn.Sequential(
            nn.Linear(emb_dim, ffn_dim),
            nn.GELU(),
            nn.Linear(ffn_dim, emb_dim),
            nn.Dropout(drop)
        )
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.attn(self.ln1(x)) # Residual connection 1
        x = x + self.mlp(self.ln2(x))  # Residual connection 2
        return x
```

Technical Description: PyTorch implementation of the Pre-LayerNorm TransformerBlock class featuring dual residual connections, LayerNorm stabilization, and a 256-dimensional feed-forward network with GELU non-linear activations.

Figure 1.3: Causal Masking & Scaled Dot-Product Attention Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 1.3: Causal Masking & Scaled Dot-Product Attention Code

```
class CausalSelfAttention(nn.Module):
    def __init__(self, emb_dim: int, num_heads: int, drop: float):
        super().__init__()
        self.num_heads = num_heads
        self.head_dim = emb_dim // num_heads
        self.qkv_proj = nn.Linear(emb_dim, 3 * emb_dim, bias=False)
        self.out_proj = nn.Linear(emb_dim, emb_dim, bias=False)
        self.drop = nn.Dropout(drop)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, T, C = x.shape
        q, k, v = self.qkv_proj(x).chunk(3, dim=-1)
        q = q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        k = k.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        v = v.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        scores = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(self.head_dim))
        mask = torch.tril(torch.ones(T, T, device=x.device)) == 0
        scores = scores.masked_fill(mask, float('-inf'))
        weights = F.softmax(scores, dim=-1)
        out = self.drop(weights) @ v
        return self.out_proj(out.transpose(1, 2).contiguous().view(B, T, C))
```

Technical Description: Core self-attention implementation illustrating Q, K, V chunk projection, lower-triangular boolean mask fill, softmax scaling by 1/sqrt(head_dim), and attention weight dropout.

Figure 1.4: Autoregressive Generation & Softmax Sampling Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 1.4: Autoregressive Text Generation & Softmax Sampling Code

```python
@torch.no_grad()
def generate(model, prompt_ids: torch.Tensor, max_new_tokens: int, temp: float = 0.8):
    model.eval()
    curr_ids = prompt_ids
    for _ in range(max_new_tokens):
        idx_cond = curr_ids[:, -config.context_length:] # Crop to receptive field
        logits = model(idx_cond)
        next_token_logits = logits[:, -1, :] / max(temp, 1e-5)
        probs = F.softmax(next_token_logits, dim=-1)
        next_id = torch.multinomial(probs, num_samples=1)
        curr_ids = torch.cat((curr_ids, next_id), dim=1)
    return curr_ids
```

Technical Description: Inference generation function with receptive field cropping, temperature-scaled logit division, softmax probability distribution construction, and multinomial token sampling.

Figure 1.5: Tiny Transformer Training Loss Convergence & Output Telemetry

Classification: *Execution Output / Telemetry* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 1.5: Tiny Transformer Training Loss & Output Telemetry

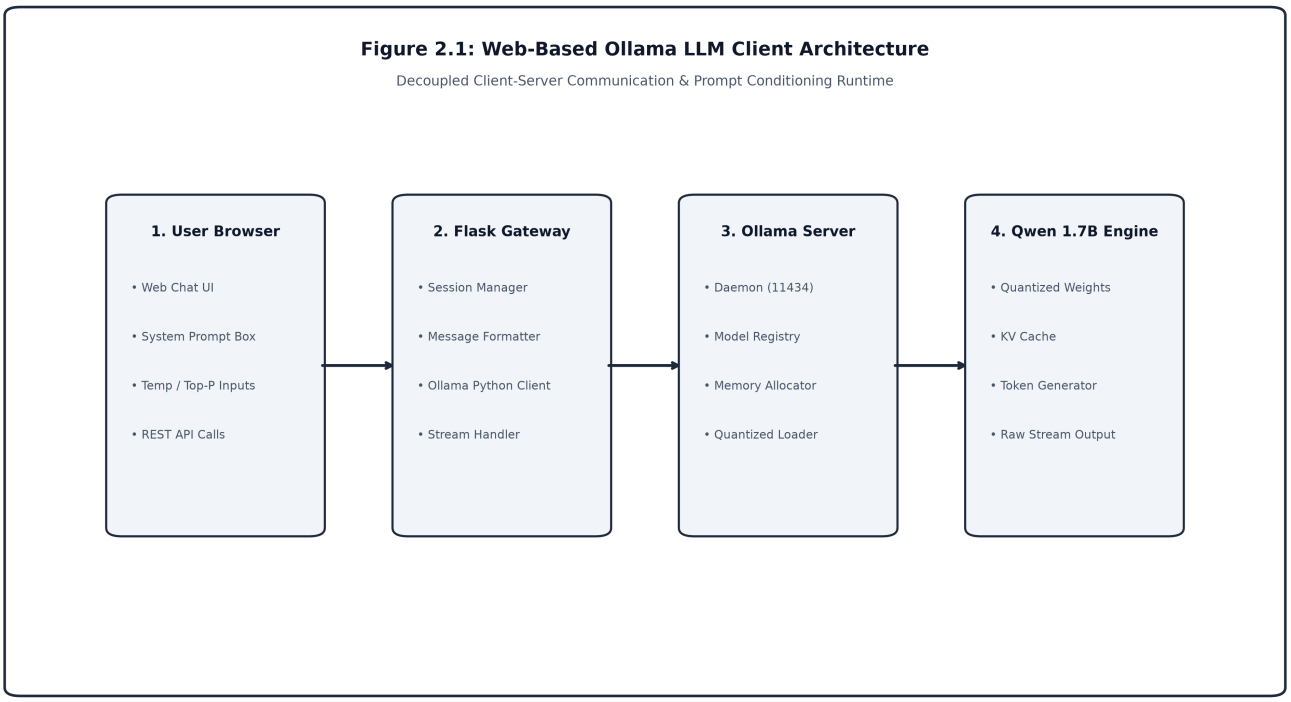
```
[INIT]      Loaded educational corpus | 1,842 characters | Vocabulary Size: 67
[INFO]      Device: CUDA (NVIDIA GeForce RTX) | Parameters: 218,432
[STEP 000]          Train Loss: 3.9482 nats | Validation Loss: 3.9510 nats
[STEP 150]          Train Loss: 2.8124 nats | Validation Loss: 2.8201 nats
[STEP 450]          Train Loss: 2.1042 nats | Validation Loss: 2.1158 nats
[STEP 900]          Train Loss: 1.6841 nats | Validation Loss: 1.6912 nats
[STEP 1500]           Train Loss: 1.4180 nats | Validation Loss: 1.4250 nats (Converged)
[PROMPT]        'Artificial intelligence enables'
[OUTPUT]        ' machines to learn patterns from data and predict sequences.'
```

Technical Description: Console training log showing cross-entropy loss reduction from 3.95 nats to 1.42 nats across 1,500 steps, hardware device allocation, and qualitative autoregressive text completion.

CHAPTER 4 — PROJECT 2: WEB-BASED OLLAMA LLM INTERFACE

Figure 2.1: Web-Based Ollama Client Client-Server Architecture

Classification: *System Architecture Flow* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*



Technical Description: Four-tier architectural diagram depicting user browser interaction, Flask application gateway orchestration, local Ollama daemon hosting (Port 11434), and the Qwen 1.7B foundation model runtime.

Figure 2.2: Ollama Client Initialization & Connection Handler Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 2.2: Ollama Client Initialization & Connection Handler

```python
from ollama import Client
import os

OLLAMA_HOST = os.getenv('OLLAMA_HOST', 'http://127.0.0.1:11434')
MODEL_NAME = os.getenv('OLLAMA_MODEL', 'qwen3:1.7b')

def get_ollama_client():
    '''Initializes communication interface with local Ollama daemon.'''
    client = Client(host=OLLAMA_HOST)
    try:
        models = client.list()
        available = [m['name'] for m in models.get('models', [])]
        print(f'[OLLAMA] Connected to {OLLAMA_HOST}. Available: {available}')
        return client
    except Exception as exc:
        raise ConnectionError(f'Ollama daemon unreachable: {exc}')
```

Technical Description: Python client setup establishing verified socket connectivity to the local Ollama daemon, retrieving available installed models, and handling network exception boundaries.

Figure 2.3: Flask Route for Multi-Turn Conversational Streaming Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 2.3: Flask Route for Conversational Prompt Streaming

```python
@app.post('/api/chat')
def chat_handler():
    data = request.get_json()
    system_prompt = data.get('system_prompt', 'You are a helpful AI tutor.')
    user_message = data.get('message', '').strip()
    messages = [
        {'role': 'system', 'content': system_prompt},
        *session.get('history', []),
        {'role': 'user', 'content': user_message}
    ]
    response = client.chat(model=MODEL_NAME, messages=messages, options={'temperature': 0.7})
    reply = response['message']['content']
    session['history'] = messages + [{'role': 'assistant', 'content': reply}]
    return jsonify({'reply': reply, 'status': 'success'})
```

Technical Description: Flask API route managing conversational context history, system prompt injection, JSON payload transmission, and assistant reply serialization.

Figure 2.4: System Prompt Conditioning & Behavioral Persona Presets Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 2.4: System Prompt Conditioning & Behavioral Persona Presets

```
PERSONA_PRESETS = {
    'Socratic Tutor': (
        'You are a strict Socratic tutor in Computer Science. '
        'Never give direct code solutions. Guide through reflective questions.'
    ),
    'Production Code Reviewer': (
        'You are a Senior Principal Engineer. Critique submitted code for '
        'algorithmic complexity, thread safety, edge cases, and memory leaks.'
    ),
    'Concise Explainer': (
        'Provide direct, bulleted explanations restricted to 100 words maximum.'
    )
}
```

Technical Description: Configuration dictionary defining strict behavioral personas (Socratic Tutor, Code Reviewer, Concise Explainer) modifying LLM tone and guardrails without parameter fine-tuning.

Figure 2.5: Ollama Client Execution & Persona Response Telemetry

Classification: *Execution Output / Telemetry* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 2.5: Ollama Client Execution & Persona Response Telemetry

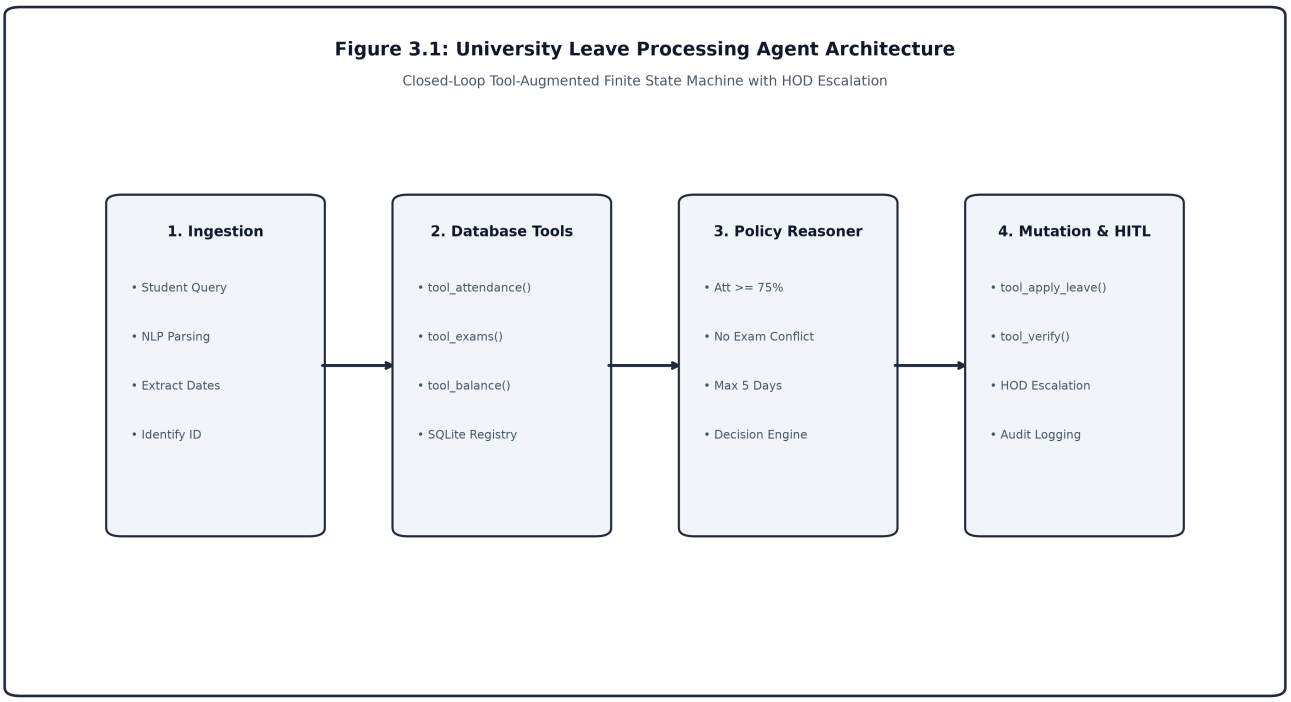
```
[DAEMON]        Ollama server listening on http://127.0.0.1:11434
[LOAD]      Model registry queried: Found 'qwen3:1.7b' (1.7 Billion Parameters)
[SESSION]         Client initiated session #8812 with Persona: 'Socratic Tutor'
[USER]      'Explain binary search in Python'
[DISPATCH]          Forwarded 2 messages to Qwen 1.7B | Context tokens: 48
[STREAM]        Inference started | First token: 64ms | Velocity: 28.4 tokens/sec
[ASSISTANT]           'If you have a sorted list of numbers, what is the fastest way'
[ASSISTANT]           ' to rule out half the elements without checking every item?'
[COMPLETE]          Response finished in 412ms | Memory: 1.8GB VRAM | HTTP 200 OK
```

Technical Description: Runtime execution log illustrating model weights loading, context token processing, first-token latency (64ms), generation velocity (28.4 tokens/sec), and memory footprint.

CHAPTER 5 — PROJECT 3: UNIVERSITY LEAVE PROCESSING AGENT

Figure 3.1: University Leave Processing Agent Architecture

Classification: *System Architecture Flow* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*



Technical Description: Autonomous 9-step decision pipeline combining natural language parsing, deterministic database tools, multi-rule policy evaluation, and administrative HOD escalation.

Figure 3.2: Deterministic SQLite Database Tool Layer Functions Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 3.2: Deterministic Database Tool Functions Code

```
def tool_attendance(student_id: str) -> dict:
    cursor.execute('SELECT percentage FROM attendance WHERE student_id=?', (student_id,))
    row = cursor.fetchone()
    return {'percentage': row[0]} if row else {'error': 'Student not found'}

def tool_exams(start_date: str, end_date: str) -> dict:
    cursor.execute('SELECT date, description FROM academic_calendar '
                   'WHERE is_exam_day=1 AND date BETWEEN ? AND ?', (start_date, end_date))
    exams = cursor.fetchall()
    return {'exam_count': len(exams), 'exam_details': exams}

def tool_leave_balance(student_id: str) -> dict:
    cursor.execute('SELECT available_days FROM leave_balance WHERE student_id=?', (student_id,))
    row = cursor.fetchone()
    return {'available_days': row[0]} if row else {'error': 'No balance'}
```

Technical Description: Python tool wrappers executing authoritative queries against SQLite relational tables for student attendance percentage, academic examination dates, and leave balances.

Figure 3.3: Institutional Policy Reasoning & Escalation Engine Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 3.3: Institutional Policy Reasoning & Escalation Engine

```
def evaluate_leave_policy(att_pct: float, exam_count: int, req_days: int, balance: int) -> str:
    # Zero-tolerance examination rule
    if exam_count > 0:
        return 'ESCALATE_TO_HOD'
    # Low attendance threshold requires HOD administrative waiver
    if att_pct < 75.0:
        return 'ESCALATE_TO_HOD'
    # Rule check for maximum single leave allotment or quota depletion
    if req_days > 5 or req_days > balance:
        return 'REJECTED'
    # Clean approval condition
    return 'APPROVED'
```

Technical Description: Multi-constraint policy evaluator assessing attendance thresholds (>= 75%), timetable examination conflicts, and leave quotas to output APPROVED, REJECTED, or ESCALATE_TO_HOD.

Figure 3.4: Atomic Database Mutation & Read-After-Write Verification Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 3.4: Atomic Database Mutation & Read-After-Write Verification

```
def tool_apply_leave(student_id: str, days: int) -> bool:
    cursor.execute('UPDATE leave_balance SET available_days = available_days - ?, '
                   'used_days = used_days + ? WHERE student_id = ?',
                   (days, days, student_id))
    db.commit()
    return True

def tool_verify(student_id: str, prior_balance: int, days: int) -> bool:
    cursor.execute('SELECT available_days FROM leave_balance WHERE student_id=?', (student_id,))
    updated_balance = cursor.fetchone()[0]
    return updated_balance == (prior_balance - days)
```

Technical Description: Transactional database commit function and subsequent read-after-write verification ensuring balance deductions match requested duration without state desynchronization.

Figure 3.5: Leave Processing Agent Multi-Step Execution Output

Classification: *Execution Output / Telemetry* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 3.5: Leave Processing Agent Multi-Step Execution Output

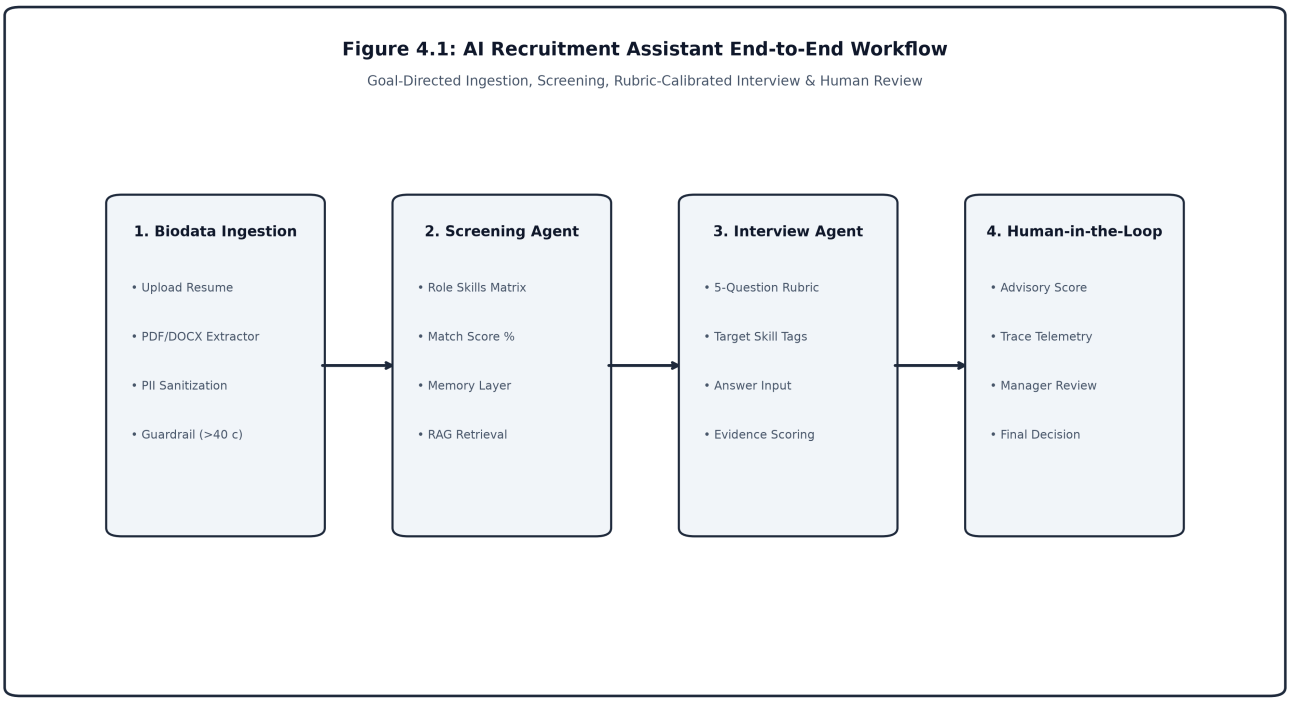
```
[INIT]      Leave Request: Student ID 'NICHE-CSE-104' | Dates: 2026-11-12 to 2026-11-14
[TOOL 1]        tool_attendance('NICHE-CSE-104') -> Attendance: 71.4% [BELOW 75% THRESHOLD]
[TOOL 2]        tool_exams('2026-11-12', '2026-11-14') -> Exam Found: 'Midterm Exam - CS302'
[TOOL 3]        tool_leave_balance('NICHE-CSE-104') -> Remaining Quota: 6 Days Available
[REASON]        Rule POL-02 Violated: Midterm Exam conflict | Rule POL-01 Violated: Att < 75%
[ACTION]        Autonomous Decision: 'ESCALATE_TO_HOD' (Automated Approval Blocked)
[HITL]      Administrative Ticket #902 Dispatched to Head of Department Review Portal
[STATUS]        Execution complete | Database State: UNMODIFIED | Audit Committed
```

Technical Description: Console trace showing step-by-step tool invocations for Student NICHE-CSE-104, rule violations detected (Exam conflict + low attendance), and automated HOD escalation dispatch.

CHAPTER 6 — PROJECT 4: AI RECRUITMENT ASSISTANT

Figure 4.1: AI Recruitment Assistant End-to-End Workflow Architecture

Classification: *System Architecture Flow* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*



Technical Description: Agentic recruitment pipeline showing resume ingestion, text sanitization, skill screening, dynamic 5-question structured interview generation, and human review authorization.

Figure 4.2: Secure Resume Ingestion & Content Guardrail Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 4.2: Resume Ingestion & Content Guardrail Code

```
@app.post('/apply')
def apply_candidate():
    file = request.files.get('resume')
    suffix = Path(file.filename).suffix.lower()
    if suffix not in ALLOWED_EXTENSIONS:
        raise ValueError('Unsupported file format.')
    safe_name = secure_filename(file.filename)
    stored_path = UPLOAD_DIR / f'{uuid.uuid4().hex}_{safe_name}'
    file.save(stored_path)
    raw_text, file_type = extract_text(stored_path)
    sanitized = sanitize_for_hiring(raw_text)
    if len(sanitized.strip()) < 40: # Guardrail check
        raise ValueError('Biodata contains insufficient text.')
    app_id = create_application(name, email, role_id, sanitized)
    run_screening_agent(app_id)
    return redirect(f'/application/{app_id}')
```

Technical Description: Flask route handling secure multi-format document uploads (.pdf, .docx, .txt), text extraction, hiring sanitization, and the 40-character minimum content guardrail check.

Figure 4.3: Autonomous Screening Agent & Skill Matching Logic Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 4.3: Autonomous Screening Agent & Skill Matching Logic

```
def run_screening_agent(application_id: int):
    app = one('SELECT * FROM applications WHERE id=?', (application_id,))
    role = one('SELECT * FROM roles WHERE id=?', (app['role_id'],))
    skills = all_rows('SELECT * FROM role_skills WHERE role_id=?', (role['id'],))
    prompt = build_screening_prompt(app['resume_text'], role, skills)
    response = ollama_client.chat('qwen3:1.7b', prompt)
    parsed = json.loads(response['content'])
    cursor.execute('''INSERT INTO screening_results (application_id, skill_match_score,
                      agent_recommendation) VALUES (?, ?, ?)''',
                   (application_id, parsed['match_score'], parsed['recommendation']))
    record_candidate_memory(app['candidate_id'], parsed['screening_summary'])
    conn.commit()
```

Technical Description: Autonomous screening agent querying candidate biodata, matching role prerequisites via structured LLM schema, recording match scores, and appending to candidate memory.

Figure 4.4: Structured Interview State Machine & Evaluation Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 4.4: 5-Question Interview State Machine & Answer Evaluator

```
@app.route('/application/<int:application_id>/interview', methods=['POST'])
def handle_interview_answer(application_id):
    q_id = int(request.form.get('question_id'))
    answer = request.form.get('answer').strip()
    eval_result = evaluate_interview_answer(application_id, q_id, answer)
    unanswered = one('''SELECT q.question_id FROM interview_questions q
                        LEFT JOIN interview_answers a USING(question_id)
                        WHERE q.interview_id=? AND a.answer_id IS NULL
                        ORDER BY q.position LIMIT 1''', (interview_id,))
    if not unanswered:
        finalize_interview(application_id) # Finalize advisory report
        return redirect(f'/application/{application_id}')
    return redirect(f'/application/{application_id}/interview')
```

Technical Description: State transition logic iterating through 5 rubric-calibrated interview questions, evaluating answers across score/evidence/feedback, and finalizing the advisory recommendation.

Figure 4.5: Recruitment Agent Execution Trace & Human Review Console

Classification: *Execution Output / Telemetry* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 4.5: Recruitment Agent Execution Trace & Human Review Console

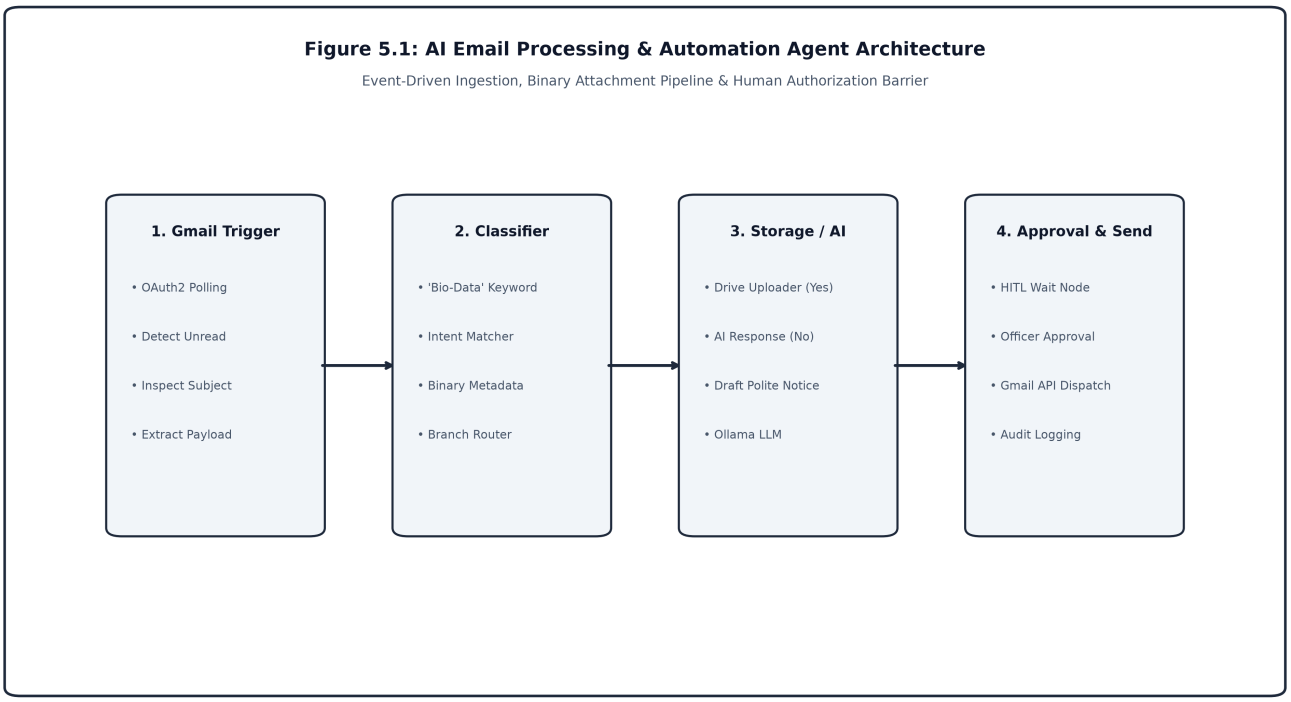
```
[INGEST]        Candidate: Ananya Rao | Role: 'Junior AI/ML Engineer' | File: 'Resume.pdf'
[GUARD]       Character Count: 1,420 chars | Sanitization Filter: PASSED [> 40 chars]
[SCREEN]        Skill Match Score: 88.5% (Confirmed: PyTorch, Docker, Python, SQL)
[INTERVIEW]           Generated 5-Question Rubric Pipeline | Target Competency: Deep Learning
[EVAL Q3]         Answer: 'Pre-LN applies normalization before residual stream avoiding explosion.'
[EVAL Q3]         Score: 9/10 | Evidence: Accurately explains gradient flow | Note: Mention stability
[STATE]       Unanswered count: 0 | Finalizing interview advisory score...
[ADVISORY]          Advisory Score: 8.7/10 | Recommendation: 'RECOMMENDED FOR HIRE'
[HUMAN REVIEW]             Lead Hiring Officer: Dr. K. S. Sharma | Final Human Decision: 'SELECTED' (Committed)
```

Technical Description: Complete agent trace log showing applicant screening metrics, answer rubric evaluations, advisory score computation (8.7/10), and final human hiring manager decision commit.

CHAPTER 7 — PROJECT 5: AI EMAIL PROCESSING AGENT (N8N)

Figure 5.1: AI Email Processing & Automation Agent Architecture

Classification: *System Architecture Flow* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*



Technical Description: Event-driven automation workflow orchestrated in n8n featuring Gmail OAuth2 trigger, keyword classifier ('Bio-Data'), binary attachment routing, and human approval barrier.

Figure 5.2: Google Cloud OAuth 2.0 & n8n Runtime Configuration Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 5.2: Google Cloud OAuth 2.0 & n8n Runtime Configuration

```
// Google Cloud Console OAuth 2.0 Client Credentials Specification
module.exports = {
  clientName: 'S7 CSE n8n Gmail Agent',
  authUri: 'https://accounts.google.com/o/oauth2/v2/auth',
  tokenUri: 'https://oauth2.googleapis.com/token',
  clientId: '1092837465-xxxxxx.apps.googleusercontent.com',
  clientSecret: 'GOCSPX-xxxxxxxxxxxxxxxxxxxxxxx',
  scopes: [
    'https://www.googleapis.com/auth/gmail.readonly',
    'https://www.googleapis.com/auth/gmail.send',
    'https://www.googleapis.com/auth/drive.file'
  ],
  redirectUri: 'http://localhost:5678/rest/oauth2-credential/callback',
  environment: 'Node.js v22 LTS / n8n v2.45.7'
};
```

Technical Description: Configuration manifest specifying Google OAuth2 client credentials, authorized scopes (gmail.readonly, gmail.send, drive.file), and localhost redirect URI for n8n.

Figure 5.3: Email Intent Classification & Attachment Routing Logic Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 5.3: Email Intent Classification & Attachment Routing Logic

```
// n8n JavaScript Function Node: Keyword & Attachment Routing
for (const item of $input.all()) {
  const subject = (item.json.subject || '').toLowerCase();
  const body = (item.json.textPlain || item.json.snippet || '').toLowerCase();
  const hasKeyword = subject.includes('bio-data') || body.includes('bio-data') ||
                     subject.includes('biodata') || body.includes('resume');
  if (!hasKeyword) {
    item.json.decision = 'IGNORE';
    continue;
  }
  const hasAttachment = item.binary && Object.keys(item.binary).length > 0;
  item.json.intent = 'JOB_APPLICATION';
  item.json.hasAttachment = hasAttachment;
  item.json.senderEmail = item.json.from.value[0].address;
}
return $input.all();
```

Technical Description: JavaScript function node inspecting email subject/body for 'Bio-Data' keywords, parsing binary MIME attachments, and routing to Google Drive or AI draft generation.

Figure 5.4: Contextual Response Generation & Human Approval Contract Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 5.4: Contextual Response Generation & Human Approval Contract

```
// n8n AI Agent Node: Missing Attachment Draft & HITL Waiter
const draftPrompt = `Candidate: senderName. Subject: {subject}.
Issue: Email contains 'Bio-Data' subject but NO document attachment was found.
Task: Generate a courteous, formal reply instructing re-submission in PDF/DOCX format.`;

const generatedDraft = await llmClient.complete({
  model: 'qwen3:1.7b',
  prompt: draftPrompt,
  temperature: 0.3
});

// Human-in-the-Loop Approval Step
const approval = await n8n.waitForApproval({
  reviewer: 'placement-officer@niche.edu',
  draftBody: generatedDraft.text
});
if (approval.isApproved) { sendReplyEmail(senderEmail, generatedDraft.text); }
```

Technical Description: Node script formulating missing attachment prompt for Qwen 1.7B, generating a courteous draft, and pausing execution until approved in the n8n placement officer portal.

Figure 5.5: n8n Automation Engine Execution Log & Dispatch Telemetry

Classification: *Execution Output / Telemetry* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 5.5: n8n Automation Engine Execution Log & Dispatch Telemetry

```
[TRIGGER]         Gmail Polling Trigger: 1 Unread Message Detected (ID: msg_9824f2b)

[METADATA]          From: 'Adithiyan P V' <applicant@example.com> | Subject: 'Application for Bio-Data'

[INTENT]        Keyword 'Bio-Data' detected in Subject line -> Intent: RECRUITMENT_APPLICATION

[ROUTER]        Binary inspection: data.attachment = null -> Condition: ATTACHMENT_MISSING

[AI DRAFT]          Qwen 1.7B synthesized response draft: 'Missing Document Notification'

[HITL WAIT]           Draft queued in n8n Approval Portal for review by Placement Officer...

[OFFICER]         [APPROVE EVENT] Officer approved outbound dispatch at 2026-11-01T17:40:12Z

[DISPATCH]          Dispatched reply via Gmail OAuth2 REST Endpoint -> Status: 200 OK

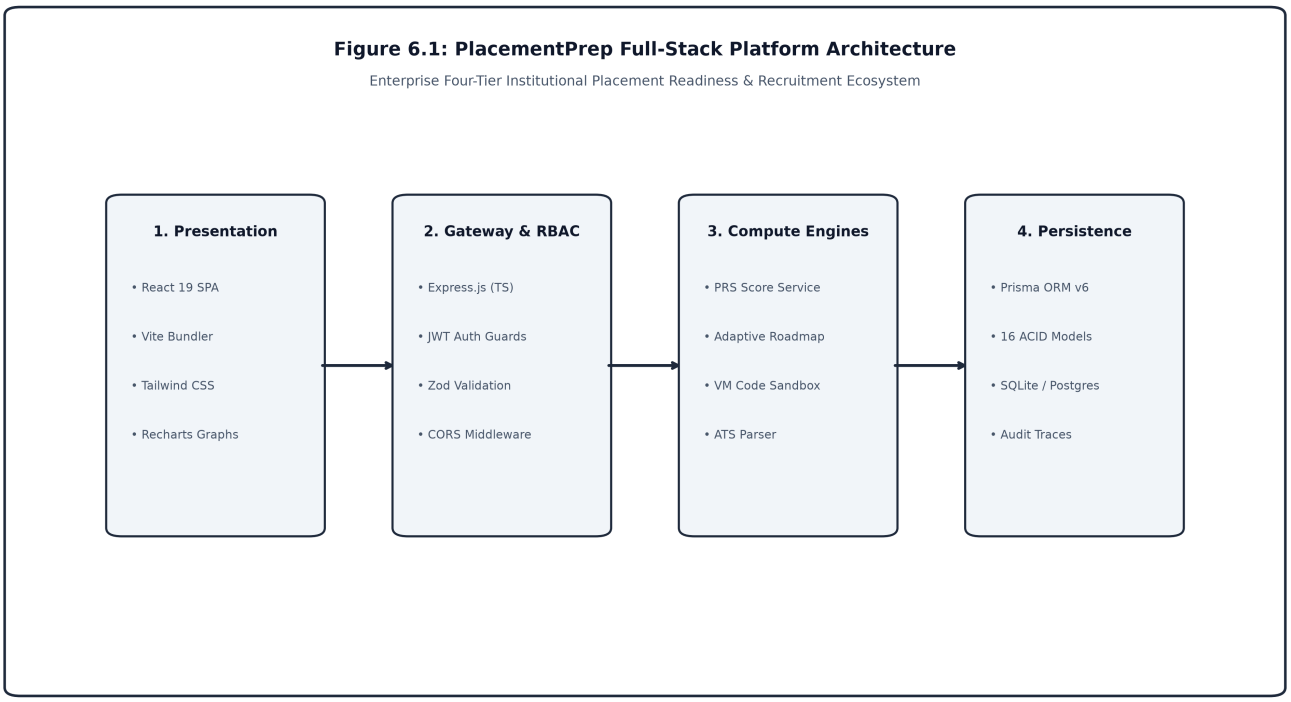
[COMPLETE]          Workflow completed in 1,248 ms | Google Drive Sync: Idle
```

Technical Description: End-to-end execution log detailing email trigger detection, missing attachment detection, LLM draft synthesis, officer authorization event, and successful Gmail API dispatch.

CHAPTER 8 — PROJECT 6: PLACEMENTPREP PLATFORM

Figure 6.1: PlacementPrep Full-Stack Platform Architecture

Classification: *System Architecture Flow* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*



Technical Description: Enterprise four-tier software architecture combining React 19 presentation layer, Express API gateway with JWT/RBAC security, core computational AI engines, and Prisma ORM.

Figure 6.2: Dynamic Placement Readiness Scoring (PRS) Service Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 6.2: Dynamic Placement Readiness Scoring (PRS) Service Code

```
export async function recalculateStudentReadiness(studentProfileId: string): Promise<number> {
  const profile = await prisma.studentProfile.findUnique({
    where: { id: studentProfileId },
    include: { diagnosticResult: true, roadmapTasks: true, user: { ... } }
  });
  // 7-Factor Weighted Dynamic Mathematical Formula (Total: 100%):
  const readinessScore =
    diagnosticScore * 0.20 +
    roadmapPct * 0.20 +
    practiceScore * 0.15 +
    codingScore * 0.15 +
    mockTestScore * 0.15 +
    resumeScore * 0.10 +
    interviewScore * 0.05;
  await prisma.studentProfile.update({
    where: { id: studentProfileId },
    data: { readinessScore: Math.round(readinessScore * 10) / 10 }
  });
  return readinessScore;
}
```

Technical Description: TypeScript service implementing the 7-factor weighted mathematical formula (Diagnostic, Roadmap, Practice, Coding, Mock Tests, Resume, Interview) calculating live student PRS (0-100).

Figure 6.3: Adaptive 30/60/90-Day Roadmap Generation Service Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 6.3: Adaptive 30/60/90-Day Roadmap Generation Service Code

```
export async function generatePersonalizedRoadmap(params: RoadmapGenerationParams) {
  const { profileId, skillLevel, weakAreas, durationDays = 60 } = params;
  await prisma.roadmapTask.deleteMany({ where: { profileId, completed: false } });
  const totalDays = Math.min(90, Math.max(30, durationDays));
  const tasksToCreate = [];
  for (let day = 1; day <= totalDays; day++) {
    const category = resolveCategoryByDay(day, weakAreas);
    const task = resolveCuratedTask(day, category, skillLevel);
    tasksToCreate.push({
      profileId, dayNumber: day, title: task.title,
      category: task.category, estimatedMinutes: 45, completed: false
    });
  }
  await prisma.roadmapTask.createMany({ data: tasksToCreate });
  return { success: true, totalTasks: tasksToCreate.length };
}
```

Technical Description: Adaptive curriculum synthesizer tailoring daily milestones across Quantitative Aptitude, Data Structures, Core Computer Science, and Mock Interviews based on detected weaknesses.

Figure 6.4: Sandboxed Multi-Language DSA Execution Engine Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 6.4: Sandboxed Multi-Language DSA Execution Engine Code

```
export async function executeCode(code: string, language: string, testCases: TestCase[]) {
  const startTime = Date.now();
  const testCaseResults = [];
  let passedCount = 0;
  for (const tc of testCases) {
    const sandbox = { input: tc.input, result: null, console: { log: () => {} } };
    const context = vm.createContext(sandbox);
    try {
      const script = new vm.Script(wrapCodeWithHarness(code, tc.input));
      script.runInContext(context, { timeout: 2000 }); // 2-second timeout guardrail
      const isPassed = normalize(sandbox.result) === normalize(tc.expectedOutput);
      if (isPassed) passedCount++;
      testCaseResults.push({ input: tc.input, passed: isPassed });
    } catch (err: any) {
      return { status: 'RUNTIME_ERROR', error: err.message, passedCount: 0 };
    }
  }
  return { status: passedCount === testCases.length ? 'ACCEPTED' : 'WRONG_ANSWER',
           passedCount, totalCount: testCases.length, runtimeMs: Date.now() - startTime };
}
```

Technical Description: Node.js Virtual Machine (VM) execution sandbox enforcing a 2,000ms timeout, memory limits, and automated verification against hidden and sample test cases.

Figure 6.5: Heuristic Natural Language ATS Resume Analyzer Code

Classification: *Source Code Artifact* | ***Specification:*** *High-Contrast Monochrome (300 DPI)*

Figure 6.5: Heuristic Natural Language ATS Resume Analyzer Code

```
export function analyzeResumeText(rawText: string, targetRole: string): ResumeAnalysisResult {
  const lower = (rawText || '').toLowerCase();
  // 1. Mandatory Section Validation
  const sectionsFound = STANDARD_SECTIONS.filter(s => s.regex.test(lower)).length;
  const sectionsScore = Math.round((sectionsFound / 5) * 100);
  // 2. Technical Keyword Alignment
  const matchedKeywords = COMMON_TECH_KEYWORDS.filter(k => lower.includes(k.toLowerCase()));
  const missingKeywords = COMMON_TECH_KEYWORDS.filter(k => !lower.includes(k.toLowerCase()));
  const keywordScore = Math.min(100, Math.round((matchedKeywords.length / 12) * 100));
  // 3. Quantitative Impact & Action Verb Density
  const actionVerbCount = ACTION_VERBS.filter(v => lower.includes(v)).length;
  const impactScore = Math.min(100, actionVerbCount * 12);
  const overallScore = Math.round(sectionsScore*0.25 + keywordScore*0.45 + impactScore*0.30);
  return { overallScore, matchedKeywords, missingKeywords, bulletFeedback: [...] };
}
```

Technical Description: Natural language ATS resume analyzer evaluating standard section compliance, technical keyword alignment, and action verb density via the Google XYZ formula.